DOSSIER PROJET CDA

HelpMeDraft

Assistant de rédaction professionnelle assistée par IA, à inférence locale

Ophélie Bellissens · Metz Numeric School · Octobre 2026

Commanditaire : LexiCorp (contexte de formation)

Déroulé

Six temps. Les numéros renvoient aux sections du dossier.

1	**Le contexte et le besoin**	Dossier § 2 et § 3
2	**La gestion de projet**	§ 4
3	**La conception**	§ 5
4	**La réalisation**	§ 6 et § 7
5	**La sécurité**	§ 8
6	**Tests, veille et bilan**	§ 9, § 10, § 11

1

Le contexte et le besoin

Dossier § 2 — le cahier des charges · § 3 — l'entreprise et le service

Le constat et la demande

LexiCorp, éditeur d'outils de gestion documentaire pour les PME, ouvre un nouveau service.

1 Le constat

Dans une PME, les écrits courants — courriels, notes, comptes rendus — mobilisent des collaborateurs qui ne sont pas rédacteurs. Qualité inégale, reprises multiples.

2 La demande

Une application web qui assiste la rédaction à l'aide d'une IA générative, dans un espace personnel sécurisé.

3 Mon rôle

Conceptrice développeuse full-stack, seule sur la chaîne : analyse, conception, front, back, base de données, sécurisation, documentation.

Les quatre objectifs du cahier des charges

Rédiger et stocker

ses documents professionnels

Suggérer et reformuler

via un modèle de langage

Un espace sécurisé

authentification et historique

Se conformer

RGPD, ANSSI, RGAA

Périmètre et exigences

Cinq blocs fonctionnels, quatre domaines d'exigences, et des limites assumées.

A–E

Périmètre fonctionnel

Gestion utilisateur — inscription, connexion, mot de passe oublié, deux rôles
Éditeur Markdown et trois commandes IA : reformuler, corriger, compléter
Gestion documentaire — CRUD, dossiers, enregistrement automatique, historique IA
Intégration IA — appels serveur, prompt engineering, affichage différencié
Back-office — comptes, statistiques, quota de 20 requêtes / 24 h

NF

Exigences non fonctionnelles

Sécurité — hachage, JWT, protection XSS / CSRF / injections
Accessibilité — RGAA 4.1.2, contrastes, clavier, ARIA
RGPD — consentement explicite, aucune donnée transmise à un tiers
Éco-conception — chargement différé, dépendances limitées, compression
AI Act — transparence, applicable depuis le 2 août 2026 (hors cahier des charges)

Hors périmètre, assumé : facturation et abonnements · collaboratif temps réel · application mobile native · export bureautique propriétaire · hébergement cloud de production.

2

La gestion de projet

Dossier § 4 — méthode, planning, versions, arbitrages

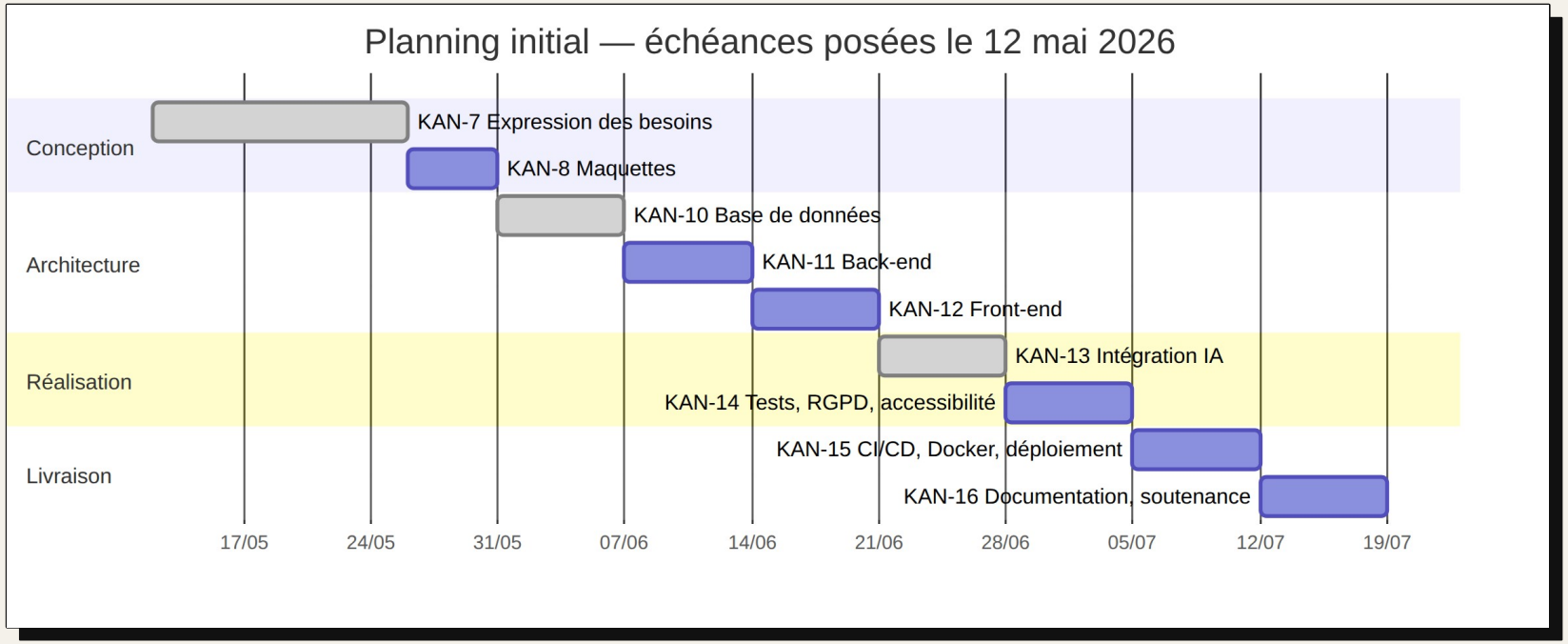
La méthode et le planning

1 semaine
durée d'une itération

6 / 6
étapes du plan engagées

13
lots fonctionnels terminés

2
points restants, documentés



Planning initial repris du cahier des charges et découpé en lots verticaux.

Les arbitrages techniques

Les trois décisions qui structurent le projet.

1 Session révocable

Un JWT seul ne se révoque pas.

Table user_session et rotation des refresh tokens : rejouer un jeton déjà consommé est traité comme un vol, et toutes les sessions du compte tombent.

2 Quota par agrégation

Comptage sur le journal des appels, pas un compteur stocké sur l'utilisateur.

La valeur ne peut pas se désynchroniser, et l'historique reste auditable — ce qui sert aussi la traçabilité.

3 Confidentialité

Inférence locale : les contenus ne quittent pas l'infrastructure. L'exigence est satisfaite par construction.

Contreparties assumées — latence, qualité, maintenance.

Les trois tiennent une exigence du cahier des charges par la structure du code, et non par une vérification ajoutée après coup.

3

La conception

Dossier § 5 — architecture, maquettes, modèle de données, cas d'utilisation

La pile technique

Comparée à l'environnement technique recommandé au § 4 du cahier des charges.

Front-end	Vue 3 · TypeScript · Vite · Tailwind 4 · Pinia	ÉCART
Éditeur	CodeMirror 6 · marked · DOMPurify	AJOUT
Back-end	Python 3.11+ · Flask 3 · SQLAlchemy 2	CONFORME
Base de données	MySQL 8.4 (SQLite en développement et en test)	AU CHOIX
IA	Ollama en local · qwen2.5:3b	AU CHOIX
Authentification	JWT HS256 · refresh httpOnly haché, en rotation	CONFORME
Tests	pytest · pytest-cov · Vitest	AJOUT
CI / CD	GitHub Actions — 3 travaux parallèles	CONFORME
Conteneurisation	Docker Compose — MySQL, gunicorn, nginx	CONFORME

CONFORME à la recommandation · **AU CHOIX** dans une alternative ouverte par le cahier des charges · **ÉCART** assumé · **AJOUT** hors cahier des charges

Trois choix de technologie

Un écart assumé, et deux alternatives que le cahier des charges ouvrait lui-même.

ÉCART

Vue 3 plutôt que React

Le cahier des charges recommande, il n'impose pas.

Même modèle de composants ; ma maîtrise y était meilleure, et seule sur un projet la maîtrise réduit le risque.

Intégration directe de CodeMirror 6.

Le découpage services / store reste transposable à React.

AU CHOIX

Ollama plutôt qu'OpenAI

Le cahier des charges autorise les deux. C'est le RGPD qui tranche.

L'anonymisation n'est jamais exhaustive sur du texte libre. En local, les contenus ne sortent pas : l'exigence est satisfaite par construction.

Contreparties assumées — latence, qualité, maintenance.

AU CHOIX

MySQL plutôt que MongoDB

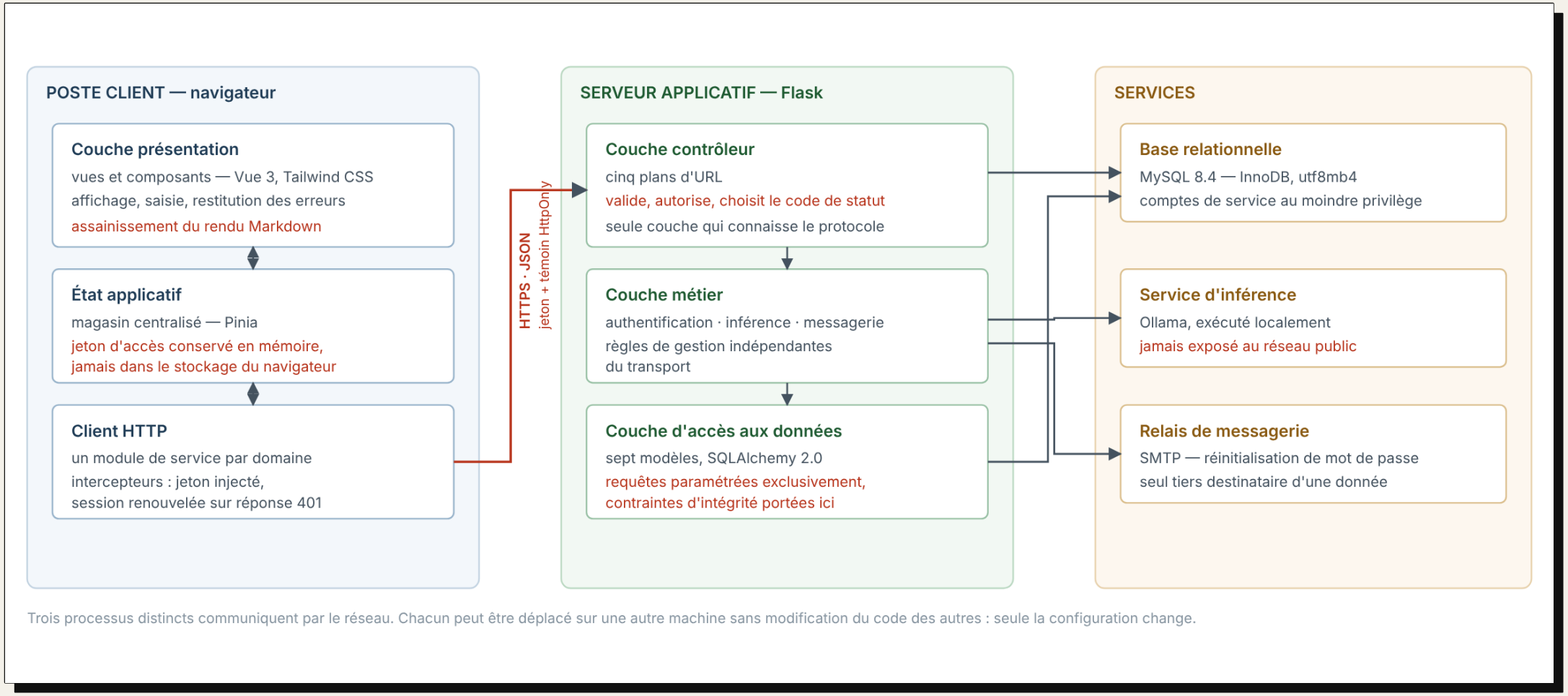
Schéma stable, données fortement reliées, volume modeste.

Les clés étrangères en ON DELETE CASCADE mettent en œuvre le droit à l'effacement en une instruction.

Aucun NoSQL ajouté : un composant qui ne sert qu'à remplir une grille est une dette déguisée.

L'argumentaire NoSQL complet est un livrable à part : docs/argumentaire-nosql.md, daté du 4 octobre.

L'architecture logicielle



Trois processus distincts communiquent par le réseau. Chacun peut être déplacé sur une autre machine sans modification du code des autres : seule la configuration change.

Architecture logicielle — trois niveaux et services périphériques (dossier § 5.2).

Le découpage en couches

La règle d'architecture qui rend le plan de tests possible.

Couche	Responsabilité	Ne contient jamais
Présentation `app/routes/`	Validation des entrées, codes de retour, sérialisation JSON	Règle métier
Métier `app/services/`	Hachage, cycle de vie des jetons, prompts, appel au modèle, courriels	Flask : request, response
Persistance `database/db.py`	Modèles SQLAlchemy, relations, contraintes, requêtes et agrégations	Logique de présentation

Côté front, la même règle : aucune vue n'appelle axios. L'intercepteur qui rejoue une requête après un 401 est donc écrit une seule fois.

L'éditeur de document



Capture C-08 — éditeur de document, écran livré (dossier § 12.2). Neuf planches de maquettage l'ont précédé (§ 12.1).

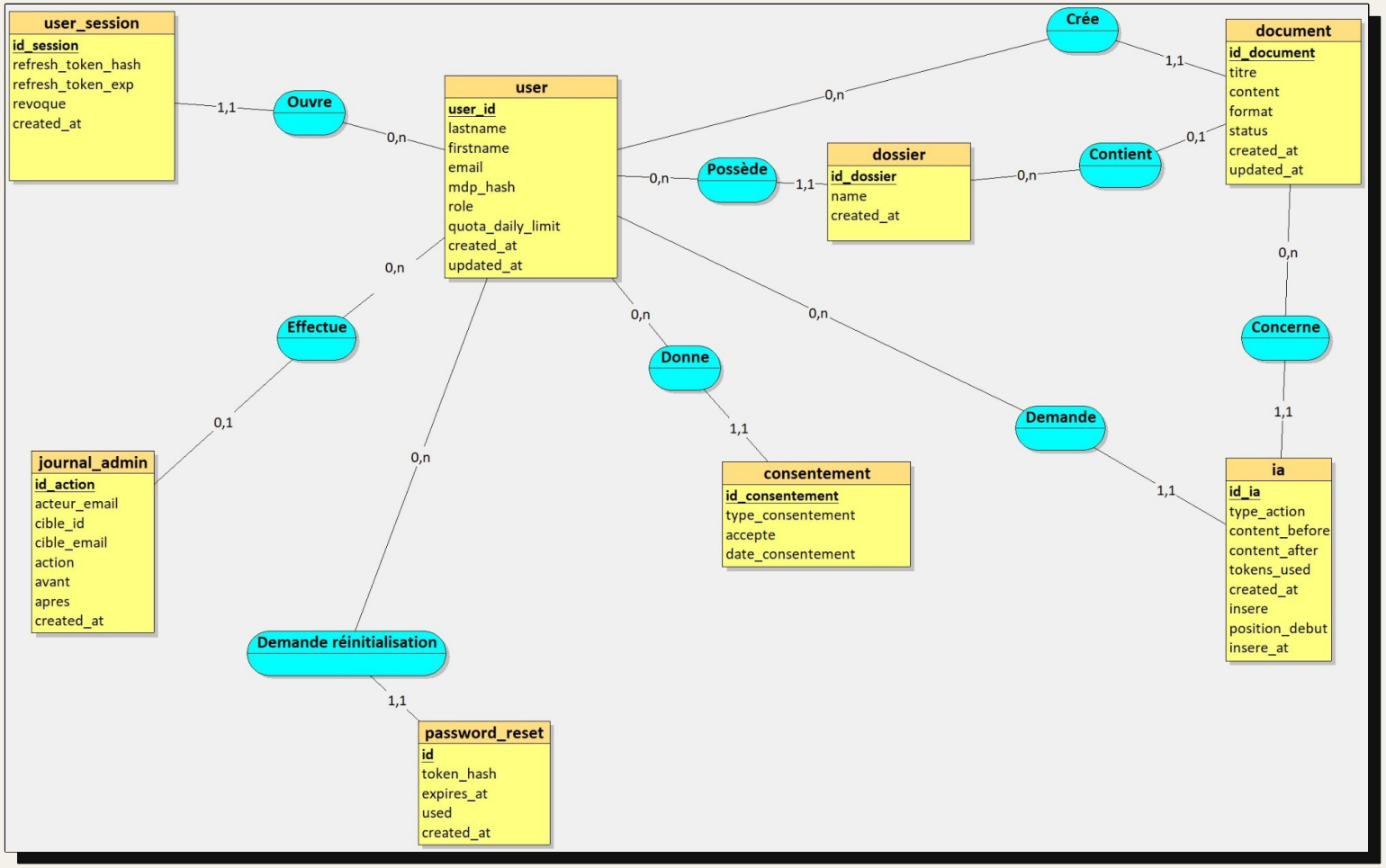
Les zones d'accès et les gardes du routeur

Seize routes déclarées dans frontend/src/index.ts, et un seul beforeEach pour les trois gardes.

Public ouvert à tous	`aucune`	/ · /fonctionnalites · /tarifs · /modeles /mentions-legales · /cgu · /confidentialite
Public — visiteur seul déjà connecté → /dashboard	`guestOnly`	/login · /register · /forgot-password · /reset-password
Espace authentifié sans jeton → /login	`requiresAuth`	/dashboard · /documents /documents/nouveau · /documents/:id
Back-office rôle ≠ admin → /dashboard	`requiresAuth + requiresAdmin`	/admin

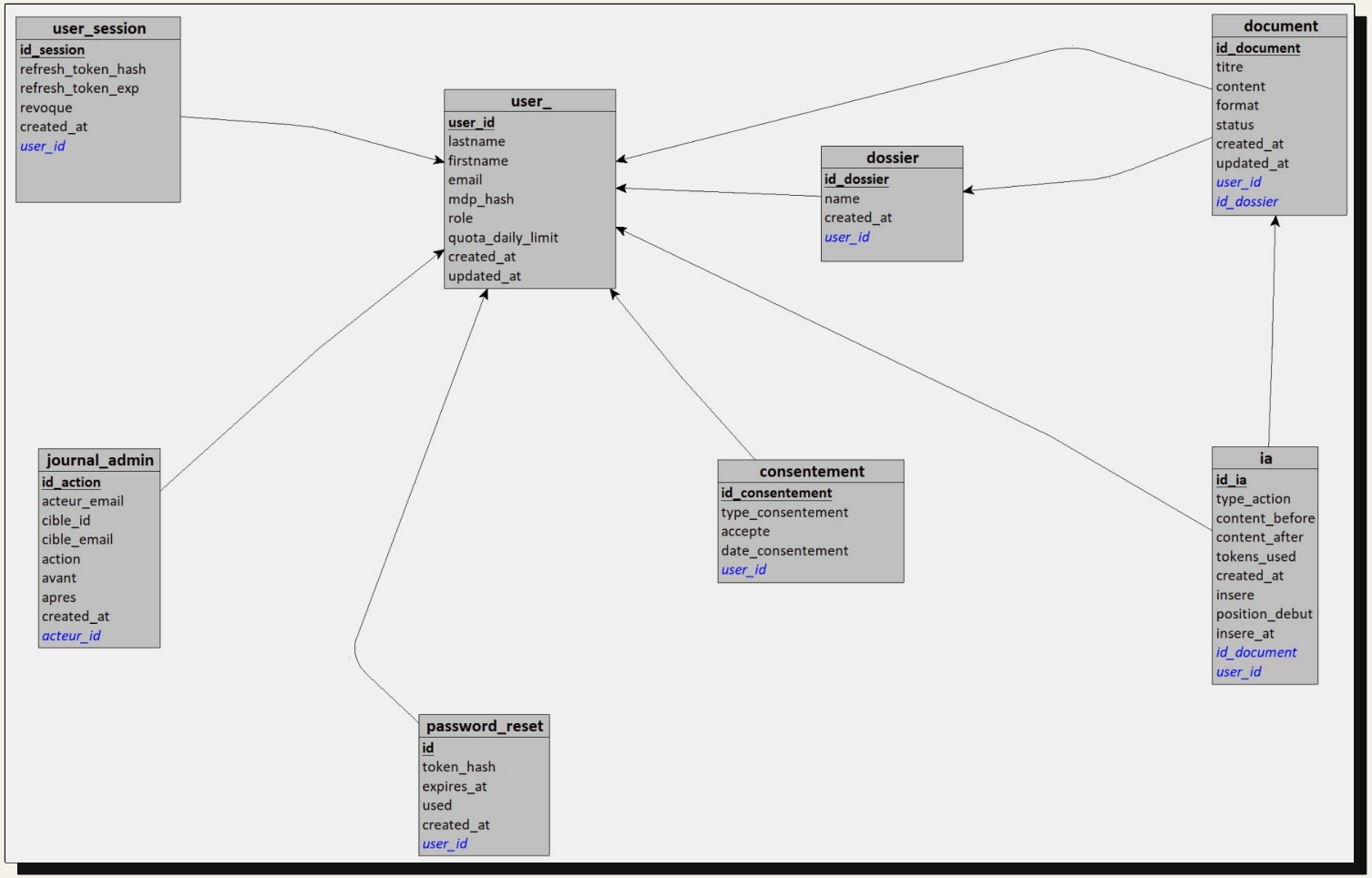
Ces trois gardes sont du confort d'usage. Elles évitent un écran vide et masquent le menu d'administration. Le contrôle qui compte est côté serveur — § 8.

Le modèle conceptuel de données



Modèle conceptuel de données (dossier § 5.4).

Le modèle logique de données



Modèle logique de données (dossier § 5.4) ; modèle physique et script MySQL en § 5.4 et § 5.5.

Les cas d'utilisation

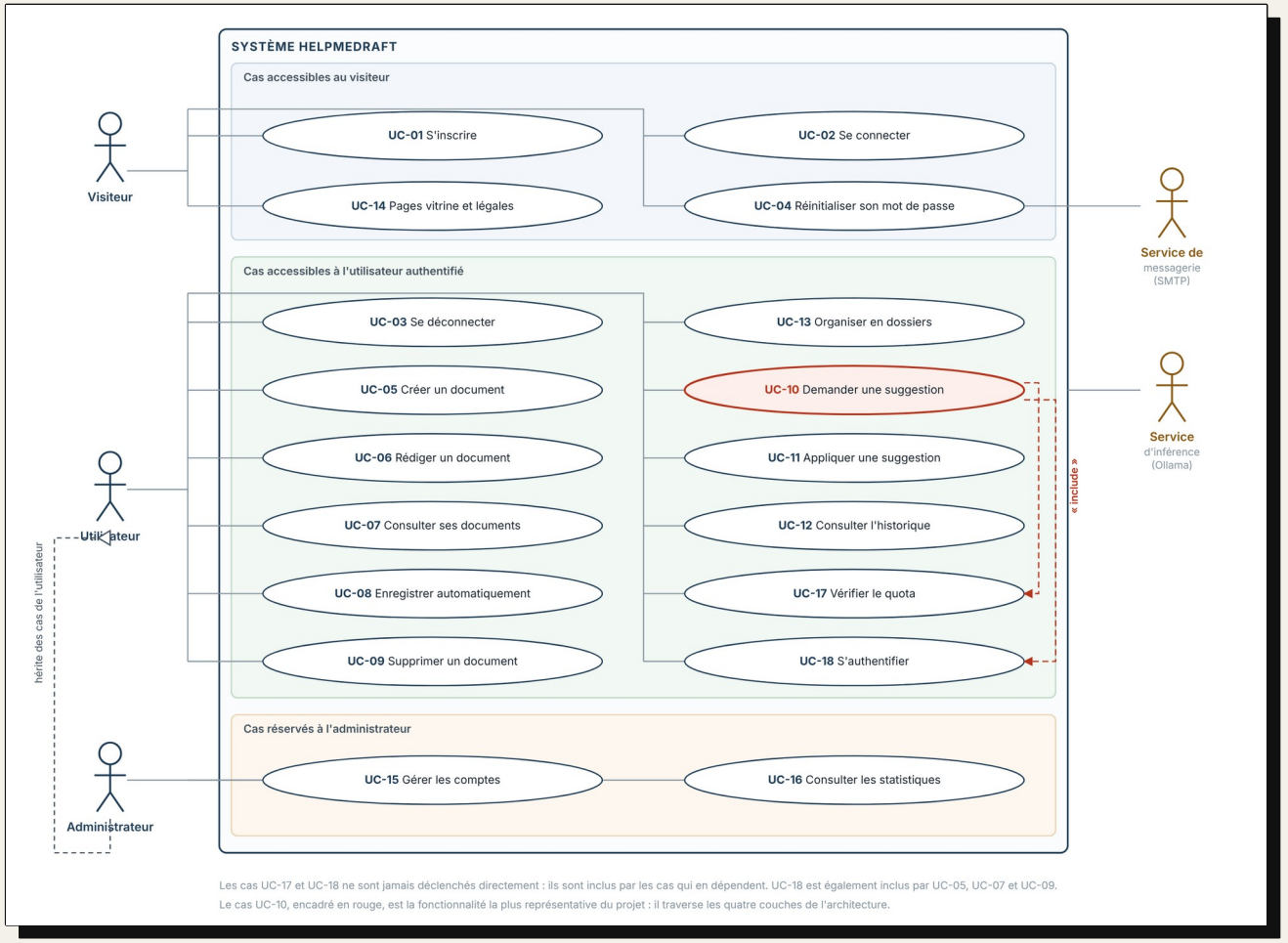
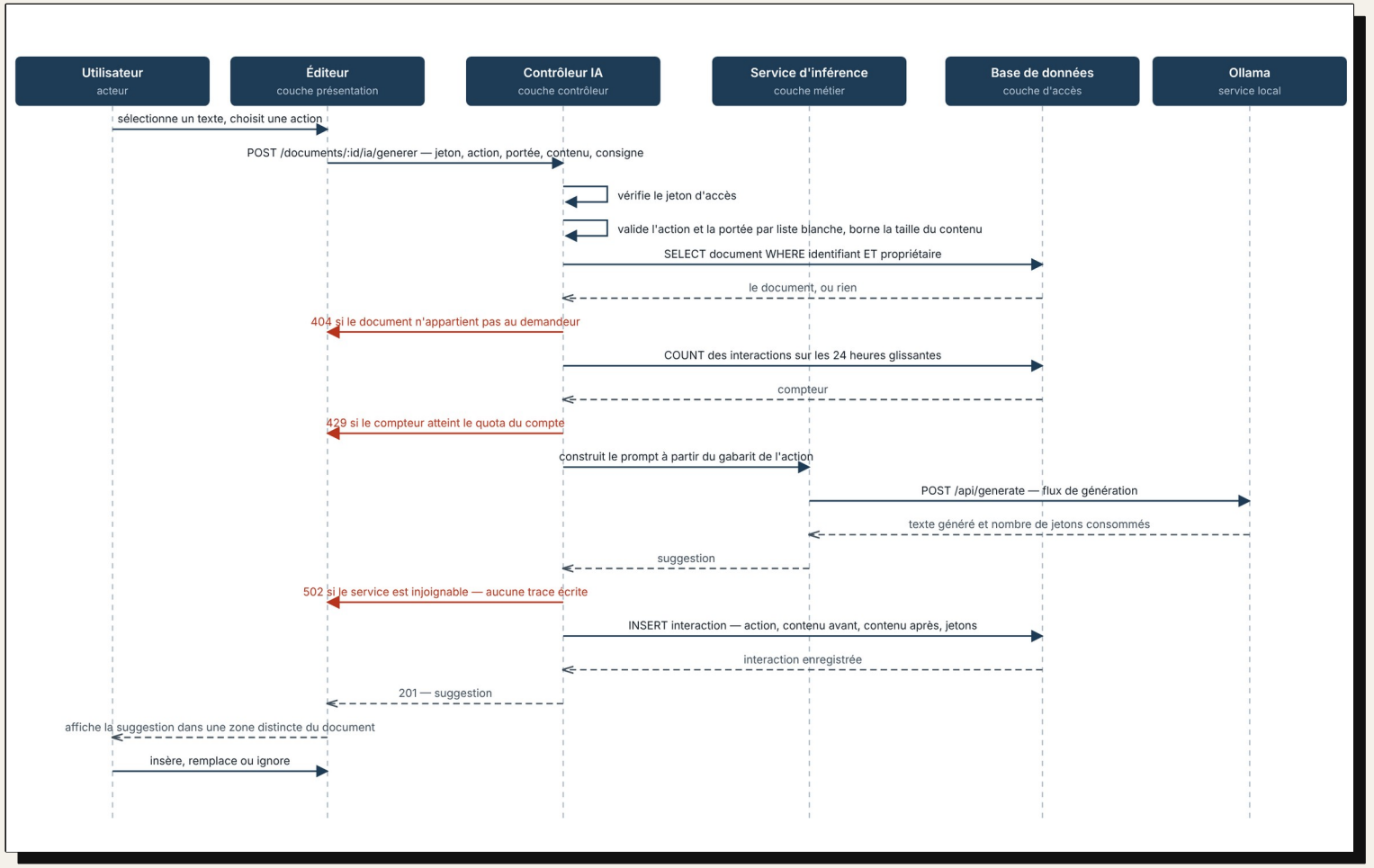


Diagramme des cas d'utilisation (dossier § 5.6).

Le diagramme de séquence — génération IA



Séquence — génération d'une assistance IA (dossier § 5.7).

4

La réalisation

Dossier § 6 — spécifications techniques · § 7 — écrans et extraits de code

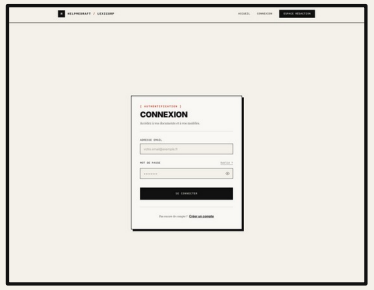
Les écrans livrés



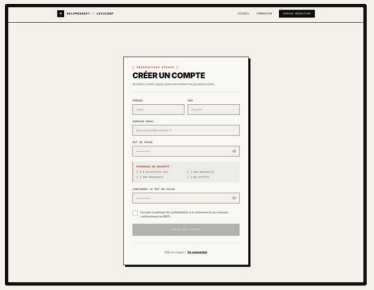
C-01 Accueil



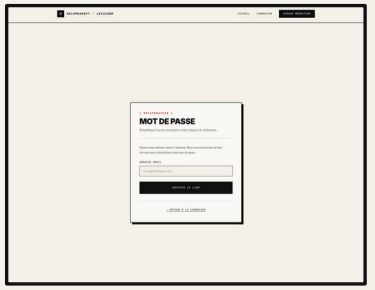
C-01b Mobile



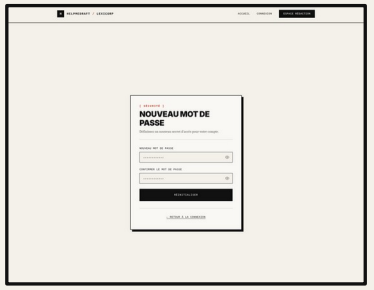
C-02 Connexion



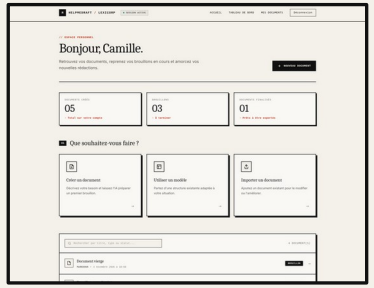
C-03 Inscription



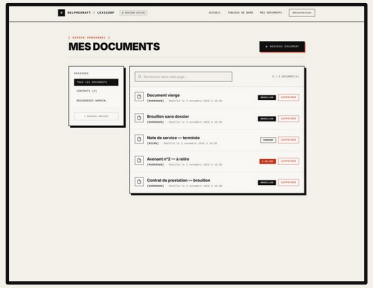
C-04 Mot de passe



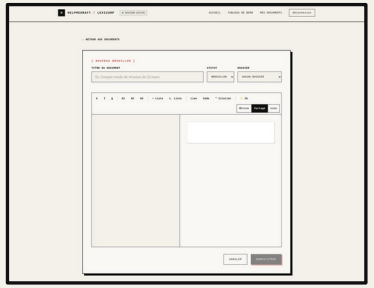
C-11 Réinitialisation



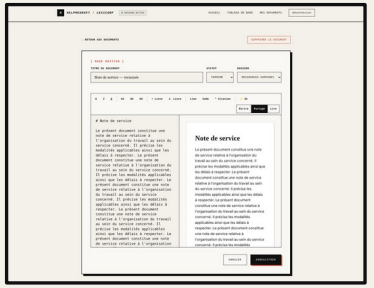
C-05 Tableau de bord



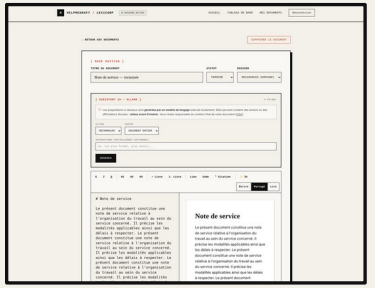
C-06 Mes documents



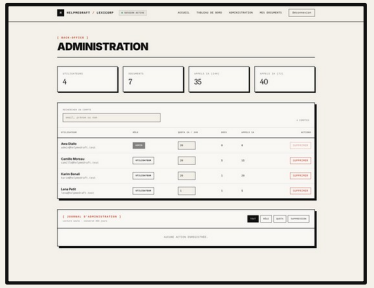
C-07 Nouveau document



C-08 Éditeur



C-09 Assistant IA



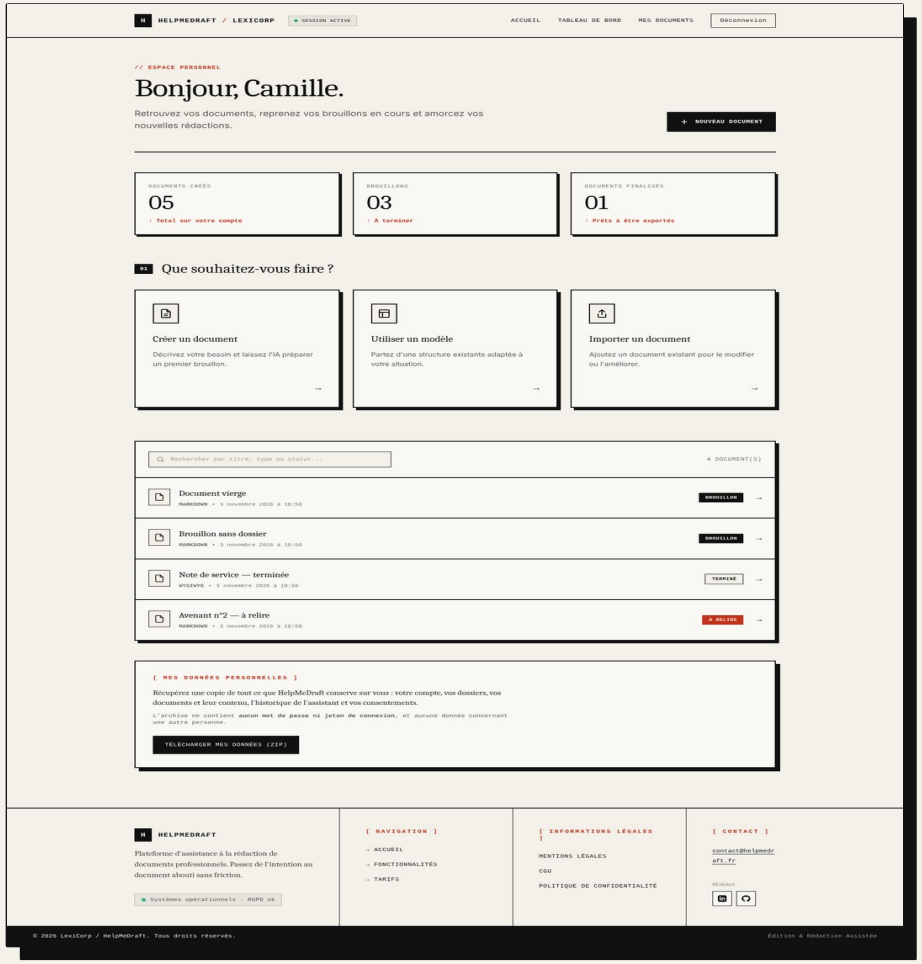
C-10 Back-office

Ligne 1 : zone publique et pages réservées au visiteur. · Ligne 2 : espace authentifié, puis back-office.

Captures d'interfaces (dossier § 12.2) — chacune est accompagnée du code correspondant.

Le tableau de bord

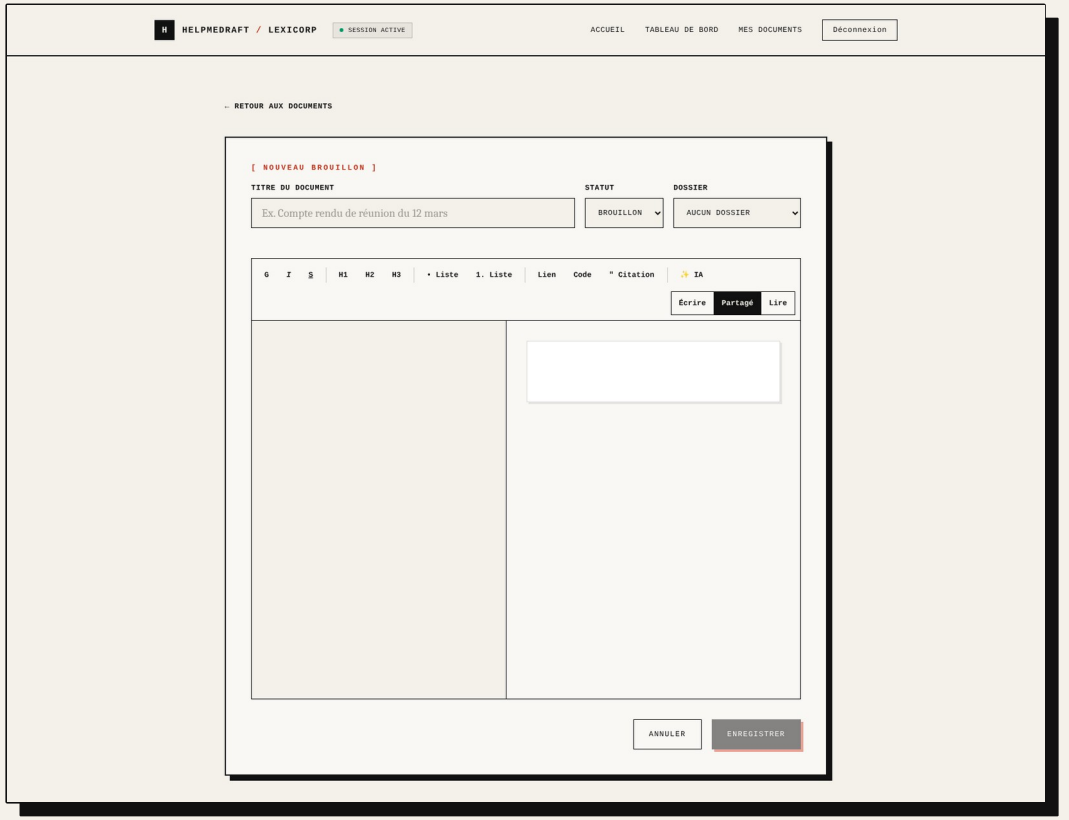
- Trois compteurs — documents créés, brouillons, finalisés — servis par l'API, jamais recalculés côté client.
- Trois portes d'entrée : page blanche, modèle, document existant. Devant un éditeur vide, l'utilisateur sans structure en tête abandonne.
- Pagination serveur : la liste n'est jamais chargée entièrement ; la recherche filtre la page courante.
- État vide explicite plutôt qu'une zone muette.
- Huit captures figurent en annexe § 12.2, avec le code qui les produit.



Capture C-05 — tableau de bord.

L'éditeur et l'enregistrement automatique

- 2,5 s : enregistrer à chaque caractère saturerait l'API ; un délai trop long ferait perdre du travail.
- Le minuteur précédent est annulé à chaque frappe — sinon les requêtes s'empilent.
- Doublé d'un bouton explicite : l'utilisateur garde la main.
- Rien n'est enregistré tant que le titre est vide.
- Une capture atteste d'un état, pas d'un comportement — d'où le jeu d'essai du § 10.



Capture C-07 — éditeur, création d'un document.

Un composant métier — le quota IA

Contrôlé côté serveur, avant tout appel au modèle (dossier § 7.2).

app/routes/ia_route.py

```
window_start = utc_now_naive() - timedelta(hours=24)

calls_last_24h = db_session.execute(
    select(func.count()).select_from(IA).where(
        IA.user_id    == request.user_id,
        IA.created_at >= window_start,
    )
).scalar_one()

if calls_last_24h >= current_user.quota_daily_limit:
    return jsonify({"error": "Quota IA atteint."}), 429
```

Le front affiche le quota restant — mais le contrôle qui compte est celui du serveur.

Une agrégation, pas un compteur

Un compteur stocké se désynchronise ; une agrégation est juste par construction.

Une fenêtre glissante

« 20 par 24 h », pas « par jour » : sinon 40 appels entre 23 h et 1 h.

429, pas 403

« Réessayez plus tard », et non « vous n'avez pas le droit ».

Un composant d'accès aux données

Le motif répété pour chaque ressource de l'application (dossier § 7.3).

app/routes/document_route.py

```
stmt = select(Document).where(
    Document.id_document == id_document,
    Document.user_id     == request.user_id,
)
document = db_session.execute(stmt).scalar_one_or_none()

if document is None:
    return jsonify({"error": "Document introuvable"}), 404
```

Filtrer, pas vérifier après

Oublier la condition fait échouer un test ; oublier un if ouvre une faille.

404, jamais 403

Un 403 confirmerait que la ressource existe. Le 404 est indiscernable.

ORM paramétré, sans exception

Aucune requête construite par concaténation dans tout le projet.

Les identifiants étant des UUID v4, ils ne sont de toute façon pas énumérables — défense en profondeur.

La conteneurisation et l'intégration continue

Trois services, trois travaux de CI, et une liste de contrôle pour une mise en ligne (dossier § 6).

3× Conteneurisation

MySQL 8.4, volume persistant et sonde de santé.
Backend sous Gunicorn, utilisateur non privilégié.
Frontend Vite puis Nginx, 82,9 Mo.

Secrets en ${VAR:?} : la composition échoue si la variable manque.

CI Intégration continue

GitHub Actions à chaque push et chaque pull request, en trois travaux parallèles : backend (Ruff, pytest), frontend (ESLint, vue-tsc, Vitest, build), Docker (composition et images).

Installation par npm ci --ignore-scripts.

→ Le jour d'une mise en ligne

APP_ENV=production, qui active Secure sur le cookie.
Une JWT_SECRET_KEY distincte.
CORS_ORIGINS au domaine réel.
Compte MySQL sans droits DDL.
TLS devant l'application.
Redis pour le limiteur de débit.

Ollama reste hors composition, sur l'hôte : son image pèse plusieurs gigaoctets et le modèle se télécharge à part.

5

La sécurité

Dossier § 8 — OWASP Top 10, sessions, contrôle d'accès, validation

Les risques OWASP et leur traitement

Grille OWASP Top 10 (2021), croisée avec le cahier des charges et les recommandations de l'ANSSI.

A01 **Contrôle d'accès**

Propriété dans le WHERE, rôle relu en base à chaque appel /admin, garde de blueprint.

A02 **Cryptographie**

bcrypt salé pour les mots de passe, SHA-256 pour les jetons. Aucun secret en clair en base.

A03 **Injection**

ORM paramétré sans exception ; rendu Markdown assaini par DOMPurify.

A05 **Configuration**

Sans JWT_SECRET_KEY, l'application refuse de démarrer. CORS restreint à l'origine du front.

A07 **Identification**

Rotation des refresh tokens, révocation, limitation de débit, politique de mot de passe.

A10 **SSRF**

Une seule URL sortante : OLLAMA_URL, valeur de configuration, jamais une entrée utilisateur.

A09 — la journalisation applicative reste incomplète : le journal IA existe, pas la journalisation systématique des événements de sécurité.

Le cycle de vie des sessions

Deux besoins contradictoires, résolus par deux jetons de nature différente (dossier § 8).

Access token

JWT HS256, 15 minutes, en-tête Authorization, jamais stocké en base.

Refresh token

64 octets aléatoires, cookie httpOnly / Secure / SameSite=Strict / path=/auth, 7 jours.

Stockage

Empreinte SHA-256 uniquement : une fuite de la base ne permet pas de rejouer une session.

app/services/auth_service.py

```
if session.revoke:
    revoke_all_user_sessions(
        session.user_id, db_session)
    db_session.commit()
    raise ValueError(
        "Réutilisation détectée")

session.revoke = True
```

Rejouer un jeton déjà consommé est traité comme un vol : toutes les sessions du compte sont révoquées. La gêne est une reconnexion ; le risque évité est un compte compromis.

À la réinitialisation du mot de passe, toutes les sessions sont également supprimées : changer son mot de passe ne sert à rien si les sessions de l'attaquant survivent.

6

Tests, veille et bilan

Dossier § 9 — plan de tests · § 10 — jeu d'essai · § 11 — veille

Le plan de tests et ses résultats

Campagne rejouée le 4 novembre 2026 — quatre niveaux, du moins coûteux au plus coûteux à exécuter.

461
cas automatisés, tous au vert

403
backend · 58 frontend

95 %
couverture backend

100 %
sur la chaîne d'authentification

Analyse statique

vue-tsc, Ruff

`à chaque push`

Tests unitaires

pytest, Vitest

`à chaque push`

Tests d'intégration

pytest + client Flask

`à chaque push`

Tests fonctionnels

cahier de tests manuel

`fin d'itération`

Trois zones concentrent l'effort : la chaîne d'authentification, le contrôle de propriété et le quota — les trois endroits où une régression coûte cher.

Le jeu d'essai

Jeu d'essai sur POST /documents/<id>/ia/generer — la fonctionnalité la plus significative du projet.

? Pourquoi cette route

Elle traverse les trois couches et un système externe, concentre cinq contrôles, et porte les deux exigences les plus délicates : quota et traçabilité.

4 Quatre comptes

U1 utilisatrice principale · U2 tiers, pour l'accès croisé · U3 administrateur · U4 quota abaissé à 1, pour atteindre la limite en un appel.

25 H La donnée décisive

19 lignes de moins de 24 h, et 1 ligne datée de 25 h. Seule façon de distinguer une fenêtre glissante d'un compteur journalier.

Ce que j'en retiens. Construire un jeu de données, ce n'est pas remplir des tables : c'est choisir les valeurs qui font échouer les mauvaises implémentations. Sans la ligne à 25 heures, le cas JE-07 passerait quelle que soit l'implémentation du quota.

Les adresses utilisent le domaine réservé .test et les mots de passe sont des valeurs de test : aucune donnée personnelle réelle dans le jeu d'essai.

La veille technologique

Huit entrées entre avril et septembre 2026, trois ont appelé une action. Journal complet : docs/veille-helpmedraft.html

5.1 DOMPurify — corrigé

CVE-2026-0540, contournement du nettoyage XSS. Version affectée : la mienne. Mise à jour en 3.4.16 le 02/10/2026.

Une mesure de sécurité est un composant comme un autre.

9.1 Ollama — corrigé

CVE-2026-7482, lecture hors limites sans authentification. Atténuée par l'écoute sur localhost seule. Mise à jour en 0.35.1.

Héberger soi-même, c'est maintenir soi-même.

UE AI Act — en cours

Transparence applicable depuis le 02/08/2026. Partiellement couvert ; trois compléments identifiés : mention au premier usage, marquage des passages générés, clause CGU.

axios, 31 mars 2026 — deux versions piégées publiées après la compromission d'un compte mainteneur npm. Projet non affecté. Pas de CVE : ce n'est pas un défaut de code, c'est le canal de distribution qui a été attaqué. C'est le fichier package-lock.json, commité, qui permet de le prouver après coup.

Le bilan au regard du cahier des charges

LES QUATRE OBJECTIFS DU CAHIER DES CHARGES

- ✓ **Rédiger et stocker**

  Éditeur Markdown, CRUD, dossiers, enregistrement automatique

- ✓ **Suggérer et reformuler**

  Trois commandes, quota 20 / 24 h, chaque appel journalisé

- ✓ **Un espace sécurisé**

  Deux jetons, sessions révocables, cloisonnement par compte

- ✓ **Se conformer**

  RGPD par l'architecture, RGAA 0 violation, AI Act tracé en base

LA SÉCURITÉ, EN SIX POINTS

- La propriété est dans le WHERE — 404, jamais 403
- Le rôle est relu en base, jamais dans le jeton
- bcrypt salé ; jetons stockés en empreinte SHA-256
- Refus de démarrer sans JWT_SECRET_KEY
- Refresh httpOnly ; un rejeu révoque le compte
- Markdown brut stocké, jamais de HTML

Merci de votre attention.

La démonstration

